

Rapport de projet BAC3

FactoDB

Gestion de données de services, matériel et factures d'une entreprise informatique

Modélisation des données, Big Data et Projet (I-ILIA-024)

Titulaire : Pr. Sidi Ahmed Mahmoudi

Assistants : Aurélie Cools, Tojo Valisoa, Nourredine Bendjelloul



TABLE DES MATIÈRES

Cahier des charges	4
Contexte et définition du problème	4
Objectif du projet	4
Périmètre du projet	4
Description fonctionnelle des besoins	4
Enveloppe budgétaire	6
Délais.....	6
Modèle conceptuel de données (MCD).....	7
Normalisation.....	8
Première forme normale 1FN.....	8
Deuxième forme normale 2FN	8
Troisième forme normale 3FN.....	8
Modèle Logique de Données (MLD)	8
Modèle Physique de Données (MPD)	10
Choix des technologies	12
Implémentation des requêtes SQL et interface	13
Explication des différents types de requête SQL utilisées	13
Création et structure de la base de données (CREATE TABLE)	13
Insertion des données initiales (INSERT).....	14
Requêtes de sélection (SELECT)	15
Requêtes de mise à jour (UPDATE)	16
Requêtes de suppression (DELETE).....	17
Applications dans le site.....	17
Page d'accueil (dashboard.php)	17
Authentification (login.php et register.php)	18
Administrateur : Page d'accueil (admin/dashboard_admin.php).....	19
Administrateur : Gestion des utilisateurs (admin/manange_users.php).....	20
Administrateur : Gestion des projets (admin/manage_projects.php)	21
Administrateur : Gestion des produits (admin/manage_products.php).....	22
Responsable de projet : Page d'accueil (manager/dashboard_manager.php)	25
Responsable de projet : Gestion d'un projet (manager/project.php?idProjet=...).....	26
Client : Page d'accueil (client/dashboard_client.php)	28
Client : Création d'un projet (client/create_project.php).....	29
Client : Consulter ses projets (client/my_projects.php)	30
Client : Consulter un projet (client/project.php)	31

Client : Génération d'une facture (client/facture.php)	34
Conclusion.....	35

CAHIER DES CHARGES

CONTEXTE ET DÉFINITION DU PROBLÈME

Une entreprise informatique désire améliorer la gestion de ses services, matériels et factures en créant une base de données centralisée.

Le but du projet est donc de développer une solution permettant de gérer efficacement toutes ces données dans une base type SQL avec des fonctionnalités spécifiques (comme l'ajout, la suppression, et la modification de produits et services, la gestion des informations liées aux clients et fournisseurs, ainsi que la génération automatique de devis et factures en fonction des services et projets en cours). Le système devra également permettre de gérer les droits d'administration afin d'assurer la sécurité des données.

Tout d'abord, l'administrateur décide de créer un projet en y spécifiant les termes du contrat. Un projet représente une initiative spécifique pour un client, par exemple la mise en place d'un service ou le développement d'un produit sur mesure. Chaque projet peut comporter plusieurs services associés.

Lors de la création d'un projet, un contrat est établi, précisant les termes et conditions, les services fournis, les délais et le budget convenu entre l'entreprise et le client.

Chaque projet peut être lié à une ou plusieurs commandes, qui représentent des demandes spécifiques de produits ou services associés au projet en cours. Chaque commande aboutit à la génération d'une facture, qui précise le montant dû pour les produits et services fournis dans le cadre de cette commande.

Les services font référence aux prestations offertes par l'entreprise. Ils peuvent inclure par exemple des tâches comme le développement de logiciels, le déploiement des systèmes, etc. Chaque service a des caractéristiques bien définies, comme son coût, sa durée, et ses spécifications techniques.

OBJECTIF DU PROJET

L'objectif principal du projet est de concevoir et de développer une base de données relationnelle, qui centralisera la gestion des services, des matériels et des factures d'une entreprise informatique. Cette base de données doit permettre une gestion optimisée des produits et services offerts par l'entreprise.

Les objectifs spécifiques du projet incluent principalement :

- La centralisation des données
- L'amélioration de la façon de gérer les produits et services
- Le suivi des commandes
- La génération automatique de devis et factures
- La gestion des droits d'administration
- L'optimisation de la productivité

PÉRIMÈTRE DU PROJET

Le projet s'adresse à une entreprise informatique souhaitant améliorer la gestion de ses services, produits et factures, via une base de données. Il s'agit d'un projet interne à destination de Monsieur Sidi Mahmoudi et l'assistante Aurélie Cools.

DESCRIPTION FONCTIONNELLE DES BESOINS

- Gestion des produits, services et projets :
 - Ajout de produits/services : Seuls les utilisateurs autorisés (administrateurs, responsable des produits, etc.) pourront ajouter de nouveaux produits, services ou projets à la base de données en fournissant des informations comme le nom, la description, le prix, le délai de réalisation, etc.
 - Modification : Les utilisateurs de l'entreprise pourront modifier les informations liées à un produit ou service existant (prix, spécifications, etc.).
 - Suppression : Un produit ou service pourra être supprimé si celui-ci n'est plus d'actualité. Les factures liées à ceux-ci seront conservées après suppression.
 - Recherche : Les utilisateurs pourront rechercher des produits ou services via un moteur de recherche intégré (tri en fonction du mot-clé, du prix, de la tendance, etc.).
- Gestion des informations clients et fournisseurs :
 - Ajout et modification des informations du clients : Les utilisateurs agréés pourront créer de nouveaux enregistrements de clients ou de fournisseurs, en précisant des détails comme le nom, les coordonnées, l'historique des transactions, etc.
 - Suivi des contrats : Le système permettra de gérer les contrats signés avec les clients et fournisseurs, en précisant la durée du contrat, les produits ou services inclus et les termes contractuels.
 - Consultation des historiques : Il sera possible de visualiser aisément l'historique des commandes passées avec un client ou les services fournis.
- Gestion des commandes :
 - Enregistrement des commandes : Les utilisateurs pourront enregistrer une commande en associant les produits et services d'un contrat lié à un projet d'une entreprise.
 - Suivi des commandes : Le système permettra de suivre l'état des commandes (e.g. en cours-livrée-annulée) en fonction des produits commandés et des contrats signés.
- Génération automatique de devis et factures :
 - Création automatique de devis : En fonction des services fournis, des produits utilisés, et des paiements, un suivi basique des factures payées ou impayées sera intégré pour chaque client.
 - Émission de factures : Le système émettra des factures (potentiellement par mail ou directement par téléchargement pdf) en s'appuyant sur les mêmes informations que celles utilisées sur les devis.
 - Informations financières : Le système enregistrera des informations financières de base telles que les montants des devis et factures, les paiements reçus ou en attente, et le statut de paiement (payé, en retard, impayé). Cependant, le système n'inclura pas de fonctionnalités de comptabilité avancée, telles que les rapprochements bancaires ou les calculs fiscaux.
- Gestion des droits d'administration :

- Gestion des rôles utilisateurs : Pour assurer une sécurisation des accès adéquate, les utilisateurs du système seront répartis en plusieurs rôles, chacun avec des droits spécifiques. Voici une description des types d'utilisateurs et de leurs accès :
- i. Administrateur (ex : Chef de projet ou Responsable technique interne) :
 - 1. Accès complet : gestion des produits, services, projets, clients, fournisseurs et utilisateurs.
 - 2. Création, modification et suppression de n'importe quelle donnée.
 - 3. Gestion des droits d'accès des autres utilisateurs.
 - 4. Validation ou annulation des commandes et projets
 - ii. Manager (ex : Responsable des ventes ou Chef de projet commercial) :
 - 1. Création, modification et suppression des offres de produits et services (sans gestion des droits des autres utilisateurs).
 - 2. Gestion des informations des clients : consultation, ajout et modification.
 - 3. Création et gestion des projets : suivi des étapes, avancement, ajout de produits ou services aux projets.
 - 4. Génération de devis et de factures.
 - iii. Client (utilisateur externe) :
 - 1. Accès à un espace dédié pour consulter les projets en cours, les factures et les devis.
 - 2. Téléchargement des factures.
 - 3. Possibilité d'initier directement une discussion avec le responsable de projet.
 - 4. Aucun accès au catalogue de produits ni aux informations des autres clients.

ENVELOPPE BUDGÉTAIRE

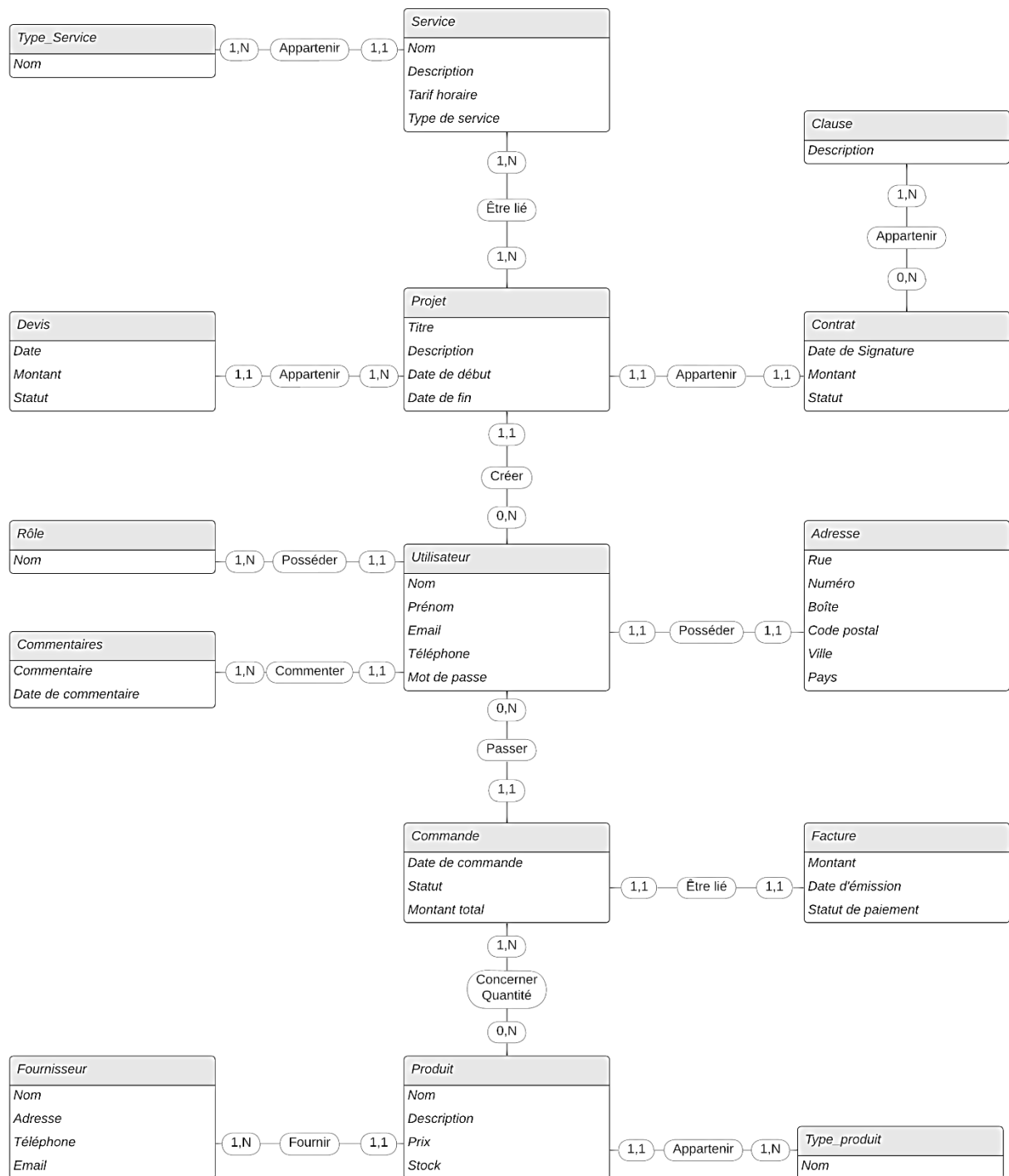
Pour la réalisation de ce projet, trois développeurs interviendront, chacun consacrant environ 63 heures de travail (séparés en 30 heures de formation et 33 heures de réalisation du projet). Le tarif horaire pour chaque développeur est fixé à 15 €, ce qui représente un coût de 945 € par développeur pour l'ensemble du travail. Ainsi, le coût total du projet pour l'équipe de développement s'élèvera à 2 835 €.

Aucune dépense supplémentaire ne sera donc nécessaire, les matériels utilisés seront personnels et le travail se fera en grande partie en distanciel.

DÉLAIS

La date de livraison du projet est fixée au mois de janvier 2025 (date exacte à déterminer). Avant cela, le cahier des charges sera remis le 6 octobre 2024. La remise du MCD et du MLD aura donc lieu le 27 novembre 2024, ainsi qu'une évaluation intermédiaire du projet le 13 décembre 2024.

MODÈLE CONCEPTUEL DE DONNÉES (MCD)



NORMALISATION

Afin d'établir une représentation conceptuelle des données optimale, il est essentiel de procéder à la normalisation de notre modèle. Plus les relations qui composent notre modèle appartiennent à une forme normale avancée, plus la qualité de notre modèle est élevée.

PREMIÈRE FORME NORMALE 1FN

Pour qu'une table soit en 1NF, chaque colonne doit contenir une valeur « atomique » (pas de valeurs multiples ou de liste dans un même champ) et il ne doit pas y avoir de groupes répétitifs

Cette première forme est vérifiée par toutes les relations de notre modèle. En effet, elles ne peuvent pas être décomposées en plusieurs attributs. De plus, nous avons veillé à ce qu'il n'y ait aucune propriété qui puisse prendre plusieurs valeurs pour un seul et même ID.

DEUXIÈME FORME NORMALE 2FN

Une table est en 2NF si elle est déjà en 1NF et que tous les attributs non-clés dépendent de la totalité de la clé primaire, et non d'une partie seulement.

- Nous devons donc créer une table de liaison « Commande_produit » afin de supprimer l'attribut « Quantité » qui dépend ici de « idCommande » et « idProduit ».

TROISIÈME FORME NORMALE 3FN

Une table est en 3NF si elle est déjà en 2NF et qu'il n'y a pas de dépendance transitive entre un attribut non-clé et la clé primaire.

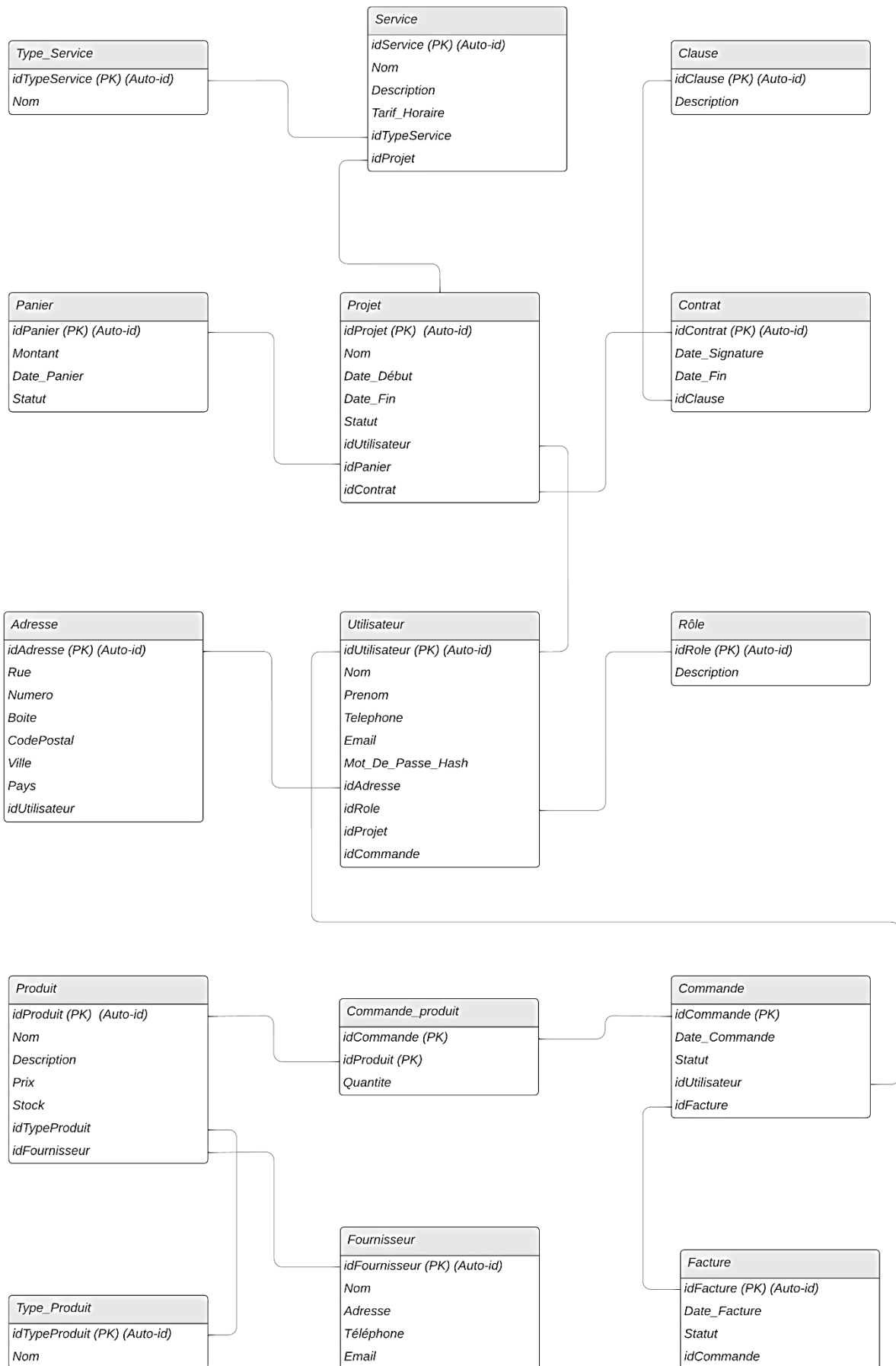
La troisième forme normale est respectée par toutes les relations présentes au sein du modèle. Ainsi, elles sont toutes en deuxième forme normale et tous les attributs qui ne composent pas la clé primaire sont mutuellement indépendants.

De plus, on ne retrouve pas de contrainte d'intégrité fonctionnelle au sein du modèle : aucune entité d'une association n'est déterminée par la connaissance d'une ou plusieurs autres entités participant à cette association.

MODELE LOGIQUE DE DONNEES (MLD)

Pour élaborer le modèle logique de notre base de données à partir du MCD, nous devons nous appliquer une série de règles :

- Chaque entité du MCD devient une relation (table) en MLD.
- Chaque association 1–N devient la clé étrangère (FK) du côté N.
- Chaque association M–N devient une relation supplémentaire (table de liaison) contenant :
 - Les deux clés étrangères (références aux entités reliées).
 - Les attributs propres à l'association (exemple : Quantité).



MODELE PHYSIQUE DE DONNEES (MPD)

Après avoir élaboré le MLD (Modèle Logique de Données), nous disposons déjà d'une vue relationnelle relativement précise : pour chaque table, nous avons identifié la clé primaire (PK), les clés étrangères (FK) pointant vers d'autres tables, ainsi que les éventuelles contraintes (unicité, clé composite, etc.). Pour passer au MPD (Modèle Physique de Données), nous devons :

1. Choisir un SGBD : MySQL.
2. Spécifier les types de données appropriés (ex. INT, VARCHAR, DATE, FLOAT, etc.).
3. Définir les contraintes d'intégrité référentielles, les règles de mise à jour/suppression (ex. ON DELETE CASCADE, ON UPDATE CASCADE), les éventuelles contraintes d'unicité et la gestion des valeurs NULL (pour les champs optionnels).
4. Écrire le code SQL qui créera physiquement les tables, leur structure et leurs relations.

Voici à titre d'exemple, un extrait du code SQL :

```
52 CREATE TABLE Role(  
53     idRole INT AUTO_INCREMENT PRIMARY KEY,  
54     Description VARCHAR(250)  
55 ) ENGINE=InnoDB;  
56  
57 CREATE TABLE Fournisseur(  
58     idFournisseur INT AUTO_INCREMENT PRIMARY KEY,  
59     Nom VARCHAR(50),  
60     Adresse VARCHAR(150),  
61     Telephone VARCHAR(15) NOT NULL,  
62     Email VARCHAR(250)  
63 ) ENGINE=InnoDB;  
64  
65 CREATE TABLE Utilisateur (  
66     idUtilisateur INT AUTO_INCREMENT PRIMARY KEY,  
67     Nom VARCHAR(50),  
68     Prenom VARCHAR(50),  
69     Telephone VARCHAR(15) NOT NULL,  
70     Email VARCHAR(250),  
71     Mot_De_Passe_Hash VARCHAR(250),  
72     idAdresse INT NULL,  
73     idRole INT NULL,  
74     idProjet INT NULL,  
75     idCommande INT NULL,  
76     CONSTRAINT unique_email UNIQUE (Email),  
77     CONSTRAINT lien_utilisateur_role FOREIGN KEY (idRole) REFERENCES Role (idRole)  
78         ON DELETE SET NULL  
79         ON UPDATE CASCADE  
80 ) ENGINE=InnoDB;
```

Une fois le code validé et les tables créées, nous avons utilisé l'onglet « Concepteur » de PhpMyAdmin afin de récupérer le Modèle Physique de Données :

CHOIX DES TECHNOLOGIES

Pour la réalisation et la gestion en tant que telle de notre base de données, nous avons utilisé MySQL ainsi que PHPMyAdmin. Nous avons choisi ces outils car il est possible de les relier entre eux et à notre site internet via un serveur Wamp.

Ensuite, nous avons programmé en HTML/CSS, PHP et JavaScript pour le site internet en lui-même. Nous avons utilisé le logiciel PHPStorm de JetBrains afin de pouvoir travailler en simultané sur le même projet via l'option « Code With Me ».

Voici un aperçu des principales technologies et outils employés :

- **PHP** : Langage de script côté serveur utilisé pour la logique métier, le traitement des formulaires, l'interaction avec la base de données et la génération dynamique de contenus (y compris la création de fichiers PDF avec Dompdf).
- **MySQL** : Système de gestion de base de données relationnelle. Il stocke toutes les données de l'application (utilisateurs, rôles, produits, projets, commandes, etc.) et permet d'effectuer des requêtes SQL pour récupérer, mettre à jour et gérer ces données de manière efficace.
- **HTML5 et CSS3** : Utilisés pour structurer et styliser l'interface utilisateur. Le HTML5 assure une sémantique claire et une bonne accessibilité, tandis que le CSS3 permet de créer des designs modernes, adaptatifs et personnalisés.
- **Bootstrap 4** : Framework CSS et JavaScript qui facilite la création de designs responsives et d'interfaces utilisateur cohérentes. Bootstrap a été utilisé pour réaliser la mise en page, les composants interactifs (navbar, formulaires, boutons) et assurer la compatibilité mobile grâce à son système de grille.
- **JavaScript / jQuery** : JavaScript, avec l'aide de la bibliothèque jQuery, a été utilisé pour enrichir l'interactivité de l'application, notamment pour les comportements dynamiques des composants Bootstrap et pour initialiser les fonctionnalités des tableaux interactifs avec DataTables.
- **DataTables** : Plugin jQuery permettant de transformer des tableaux HTML statiques en tableaux interactifs. Il offre des fonctionnalités de tri, de recherche, de pagination et de responsive design, améliorant ainsi l'expérience utilisateur lors de la consultation de grandes quantités de données.
- **Dompdf** : Bibliothèque PHP qui convertit du HTML et du CSS en documents PDF. Dans ce projet, elle a été utilisée pour générer automatiquement des factures au format PDF, permettant aux clients de télécharger et d'imprimer des factures détaillées avec des informations de paiement.
- **WampServer** : Environnement de développement web pour Windows regroupant Apache, MySQL et PHP. Il a été utilisé pour héberger localement l'application, faciliter le développement, les tests et la gestion de la base de données dans un environnement intégré.

Pour utiliser les différents fichiers de code que nous avons implémenté, il suffit de placer le dossier FactoDB dans le dossier C:\wamp64\www et ensuite d'utiliser le lien suivant pour se connecter au site : <http://localhost/FactoDB/dashboard.php>.

IMPLEMENTATION DES REQUETES SQL ET INTERFACE

Dans ce projet, nous avons conçu et manipulé une base de données relationnelle permettant de gérer différents aspects d'une entreprise informatique : rôles des utilisateurs, produits, projets, paniers, services, commandes, factures, etc. Cette base de données alimente l'application web, qui exécute différentes requêtes SQL pour réaliser toutes les opérations : lecture, insertion, mise à jour et suppression de données. Nous détaillons ci-dessous les principales catégories de requêtes et leurs rôles.

EXPLICATION DES DIFFERENTS TYPES DE REQUETE SQL UTILITEES

CREATION ET STRUCTURE DE LA BASE DE DONNEES (CREATE TABLE)

Bien que nous ayons déjà vu la structure de la base de données, voici un rappel succinct des commandes principales utilisées pour la création des tables :

```
52 CREATE TABLE Role(  
53     idRole INT AUTO_INCREMENT PRIMARY KEY,  
54     Description VARCHAR(250)  
55 ) ENGINE=InnoDB;  
56  
57 CREATE TABLE Fournisseur(  
58     idFournisseur INT AUTO_INCREMENT PRIMARY KEY,  
59     Nom VARCHAR(50),  
60     Adresse VARCHAR(150),  
61     Telephone VARCHAR(15) NOT NULL,  
62     Email VARCHAR(250)  
63 ) ENGINE=InnoDB;  
64  
65 CREATE TABLE Utilisateur (  
66     idUtilisateur INT AUTO_INCREMENT PRIMARY KEY,  
67     Nom VARCHAR(50),  
68     Prenom VARCHAR(50),  
69     Telephone VARCHAR(15) NOT NULL,  
70     Email VARCHAR(250),  
71     Mot_De_Passe_Hash VARCHAR(250),  
72     idAdresse INT NULL,  
73     idRole INT NULL,  
74     idProjet INT NULL,  
75     idCommande INT NULL,  
76     CONSTRAINT unique_email UNIQUE (Email),  
77     CONSTRAINT lien_utilisateur_role FOREIGN KEY (idRole) REFERENCES Role (idRole)  
78         ON DELETE SET NULL  
79         ON UPDATE CASCADE  
80 ) ENGINE=InnoDB;
```

Chaque table suit la même logique : un **PRIMARY KEY** sur un champ auto-incrémenté ou unique, des **FOREIGN KEY** pour assurer l'intégrité référentielle, et des types de champs adaptés aux données (VARCHAR, DATE, INT, etc.).

Ces requêtes de création incluent des options ENGINE=InnoDB pour activer les contraintes de clés étrangères et assurer l'intégrité référentielle (p. ex. ON DELETE CASCADE, ON DELETE SET NULL, etc.).

L'objectif étant de garantir la bonne correspondance des tables entre elles et assurer la cohérence de la base de données (pas de rôle inexistant, pas d'utilisateur orphelin, etc.).

INSERTION DES DONNEES INITIALES (INSERT)

Nous avons inséré les différents rôles (Administrateur, Responsable de projet, Client) dans la table Role :

```
217 INSERT INTO Role (Description)
218 VALUES
219 (Description "Administrateur"),
220 (Description "Responsable de projet"),
221 (Description "Client");
```

Ensuite, pour la table Utilisateur, nous ne pouvions pas passer par l'implémentation en SQL. En effet, notre site disposant d'un système de mots de passes cryptés, nous devons les créer manuellement un par un via la page « register.php » :

```
$passwordHash = password_hash($password, $algo: PASSWORD_BCRYPT);
$defaultRoleId = 3; // Client par défaut

$stmt = $connection->prepare( query: "
    INSERT INTO Utilisateur (Email, Mot_De_Passe_Hash, Nom, Prenom, Telephone, idRole)
    VALUES (?, ?, ?, ?, ?, ?)
");
$stmt->bind_param( types: "sssssi", &var1: $email, &var2: $passwordHash, $nom, $prenom, $telephone, $defaultRoleId);
```

Chaque compte créé se voit attribué par défaut le rôle « Client » et les valeurs à insérer sont liées au ? via la méthode bind_param, évitant toute concaténation directe de chaînes. Il nous suffit alors de modifier le rôle du premier utilisateur, qui sera un compte administrateur, via une requête SQL :

```
280 UPDATE utilisateur
281 SET idRole = 1;
282 WHERE idUtilisateur = 1;
```

Une fois un administrateur créé, il pourra ensuite modifier par lui-même les rôles des autres utilisateurs depuis l'onglet « Gestion des utilisateurs » du site :

FactoDB Tableau de Bord Se Déconnecter

Éditer l'utilisateur

Modifier les Détails de l'utilisateur

Nom	Prénom
client	Client

Adresse email

Mot de passe (Laisser vide pour ne pas changer)

Rôle

Client

Sélectionnez un rôle

Administrateur

Responsable de projet

Client

Nous avons ensuite inséré les autres données avec une requête INSERT, comme ici pour les produits :

```
INSERT INTO Produit
(Nom, Description, Prix, Stock, idTypeProduit, idFournisseur)
VALUES
( Nom 'PC Portable', Description 'PC portable pour taches communes', Prix 700, Stock 10, idTypeProduit 1, idFournisseur 1),
( Nom 'PC Portable Pro', Description 'PC portable haute performance', Prix 1200, Stock 50, idTypeProduit 1, idFournisseur 1),
( Nom 'Windows Server Licence', Description 'Licence Windows Server 2022', Prix 800, Stock 30, idTypeProduit 2, idFournisseur 2),
( Nom 'Pack Bureau', Description 'Suite complète de bureautique', Prix 200, Stock 100, idTypeProduit 2, idFournisseur 3),
( Nom 'Souris Gaming', Description 'Souris optique haute précision', Prix 40, Stock 75, idTypeProduit 4, idFournisseur 3),
( Nom 'Switch Réseau 24 ports', Description 'Switch Gigabit 24 ports', Prix 150, Stock 20, idTypeProduit 5, idFournisseur 3),
( Nom 'PC Bureau Standard', Description 'Ordinateur fixe entrée de gamme', Prix 500, Stock 60, idTypeProduit 1, idFournisseur 4),
( Nom 'Clavier Mécanique', Description 'Clavier mécanique haute durabilité', Prix 70, Stock 45, idTypeProduit 4, idFournisseur 3),
( Nom 'Logiciel CRM', Description 'CRM pour gestion de clients', Prix 300, Stock 25, idTypeProduit 2, idFournisseur 2),
( Nom 'Casque Audio', Description 'Casque pour visioconférence', Prix 25, Stock 80, idTypeProduit 4, idFournisseur 1),
( Nom 'Routeur Wi-Fi Pro', Description 'Routeur Dual-Band haute performance', Prix 120, Stock 15, idTypeProduit 5, idFournisseur 3),
( Nom 'Bureau', Description 'Bureau en bois massif pour ordinateur fixe', Prix 700, Stock 20, idTypeProduit 3, idFournisseur 4),
( Nom 'Chaise bureau', Description 'Chaise ergonomique pour bureau', Prix 300, Stock 50, idTypeProduit 3, idFournisseur 4);
```

REQUETES DE SELECTION (SELECT)

- Exemple 1 : Récupération d'un utilisateur pour la connexion dans login.php :

```
$stmt = $connection->prepare( query: "
    SELECT u.idUtilisateur, u.Mot_De_Passe_Hash, r.idRole, r.Description
    FROM Utilisateur u
    JOIN Role r ON u.idRole = r.idRole
    WHERE u.Email = ?
");
$stmt->bind_param( types: "s", &var1: $email);
$stmt->execute();
$stmt->store_result();
```

L'objectif de cette requête est de vérifier l'existence de l'utilisateur et récupérer son mot de passe haché ainsi que son rôle.

- Exemple 2 : Récupération de la liste des produits dans manage_products.php :

```
$stmt = $connection->prepare( query: "
    SELECT
        p.idProduit,
        p.Nom AS ProduitNom,
        p.Description,
        p.Prix,
        p.Stock,
        tp.Nom AS TypeProduit,
        f.Nom AS Fournisseur
    FROM Produit p
    LEFT JOIN Type_produit tp ON p.idTypeProduit = tp.idTypeProduit
    LEFT JOIN Fournisseur f ON p.idFournisseur = f.idFournisseur
    ORDER BY p.idProduit DESC
");
$stmt->execute();
$result = $stmt->get_result();
```

L'objectif de cette requête est d'afficher tous les produits dans un tableau (avec DataTables) en joignant les types de produits et les fournisseurs pour afficher un nom plus lisible :

Liste des Produits							
ID	Nom	Description	Prix (€)	Stock	Type de Produit	Fournisseur	Actions
14	Routeur Wi-Fi Pro	Routeur Dual-Band haute performance	120,00	15	Réseaux	MatLog SA	Éditer Supprimer
13	Casque Audio	Casque pour visioconférence	25,00	80	Accessoires	TechPlanet	Éditer Supprimer
12	Logiciel CRM	CRM pour gestion de clients	300,00	25	Logiciels	GlobalSoft	Éditer Supprimer
11	Clavier Mécanique	Clavier mécanique haute durabilité	70,00	45	Accessoires	MatLog SA	Éditer Supprimer
10	PC Bureau Standard	Ordinateur fixe entrée de gamme	500,00	60	Informatique	Meubles SPRL	Éditer Supprimer

REQUETES DE MISE A JOUR (UPDATE)

- Exemple 1 : Mise à jour le statut d'un projet dans manager/project.php :

```
// Mettre à jour le projet
$stmt = $connection->prepare( query: "
    UPDATE Projet
    SET Nom = ?, Description = ?, Statut = ?, Date_Debut = ?
    WHERE idProjet = ?
");
$stmt->bind_param( types: "ssssi", &var1: $new_nom, &var2: $new_description, $new_statut, $new_date_debut, $project_id);
if ($stmt->execute()) {
    $edit_message = "Projet mis à jour avec succès.";
    $edit_success = true;
}
```

L'objectif ici est que lorsque le manager valide le devis, changer le statut du projet en « Devis Envoyé ».

- Exemple 2 : Diminution du stock après ajout au panier, toujours dans manager/project.php :

```
if ($nouvelle_quantite > 0) {
    // Mettre à jour la quantité dans le panier si elle reste positive
    $stmt = $connection->prepare( query: "UPDATE Panier_produit SET Quantite = ? WHERE idPanier = ? AND idProduit = ?");
    $stmt->bind_param( types: "iii", &var1: $nouvelle_quantite, &var2: $panier_id, $product_id);
    if ($stmt->execute()) {
        // $edit_message = "Quantité mise à jour dans le panier.";
    } else {
        $edit_message = "Erreur lors de la mise à jour du panier : " . htmlspecialchars($stmt->error);
    }
    $stmt->close();
}
```

REQUETES DE SUPPRESSION (DELETE)

En reprenant l'exemple précédent, si le stock atteint 0, on supprime la ligne de produit de la base de données :

```
} else {  
    // Supprimer l'entrée du panier si la quantité atteint 0  
    $stmt = $connection->prepare("DELETE FROM Panier_produit WHERE idPanier = ? AND idProduit = ?");  
    $stmt->bind_param("ii", &var1: $panier_id, &var2: $product_id);  
    if ($stmt->execute()) {  
        // $edit_message = "Produit retiré du panier.";  
    } else {  
        $edit_message = "Erreur lors de la suppression du produit du panier : " . htmlspecialchars($stmt->error);  
    }  
    $stmt->close();  
}
```

APPLICATIONS DANS LE SITE

Nous allons à présent décrire chaque grande étape ou fonctionnalité de l'application, en explicitant les requêtes SQL (ou séries de requêtes) qui se cachent derrière.

PAGE D'ACCUEIL (DASHBOARD.PHP)

FactoDB

[Se connecter](#)

Bienvenue chez FactoDB

FactoDB est votre partenaire de confiance pour la gestion de projets, le suivi de vos commandes et la gestion de vos données informatiques. Notre équipe met en place des solutions sur mesure afin de répondre à tous vos besoins, du plus simple au plus complexe.

[Passer une commande](#)

1. Affichage initial
 - ❖ Aucune requête SQL spécifique dans cette page d'accueil, à moins d'afficher des statistiques globales.
 - ❖ But : Guider l'utilisateur vers la connexion ou l'inscription en utilisant les boutons « Passer une commande » ou « Se connecter », qui renvoient vers login.php.

AUTHENTIFICATION (LOGIN.PHP ET REGISTER.PHP)

Connexion

Email

Mot de passe

Se connecter

Pas encore de compte ? [Inscrivez-vous ici](#)

Inscription

Email

Mot de passe

Confirmer mot de passe

Prénom

Nom

Téléphone

S'inscrire

Déjà inscrit ? [Connectez-vous ici](#)

1. Connexion

- ❖ Requête **SELECT** pour vérifier l'existence de l'email et récupérer le mot de passe haché, ainsi que le rôle de l'utilisateur :

```
$stmt = $connection->prepare( query: "
    SELECT u.idUtilisateur, u.Mot_De_Passe_Hash, r.idRole, r.Description
    FROM Utilisateur u
    JOIN Role r ON u.idRole = r.idRole
    WHERE u.Email = ?
");
```

- ❖ Vérification : On compare ensuite le mot de passe fourni (haché) avec Mot_De_Passe_Hash.

2. Inscription

- ❖ Requête **SELECT** pour vérifier si l'email existe déjà :

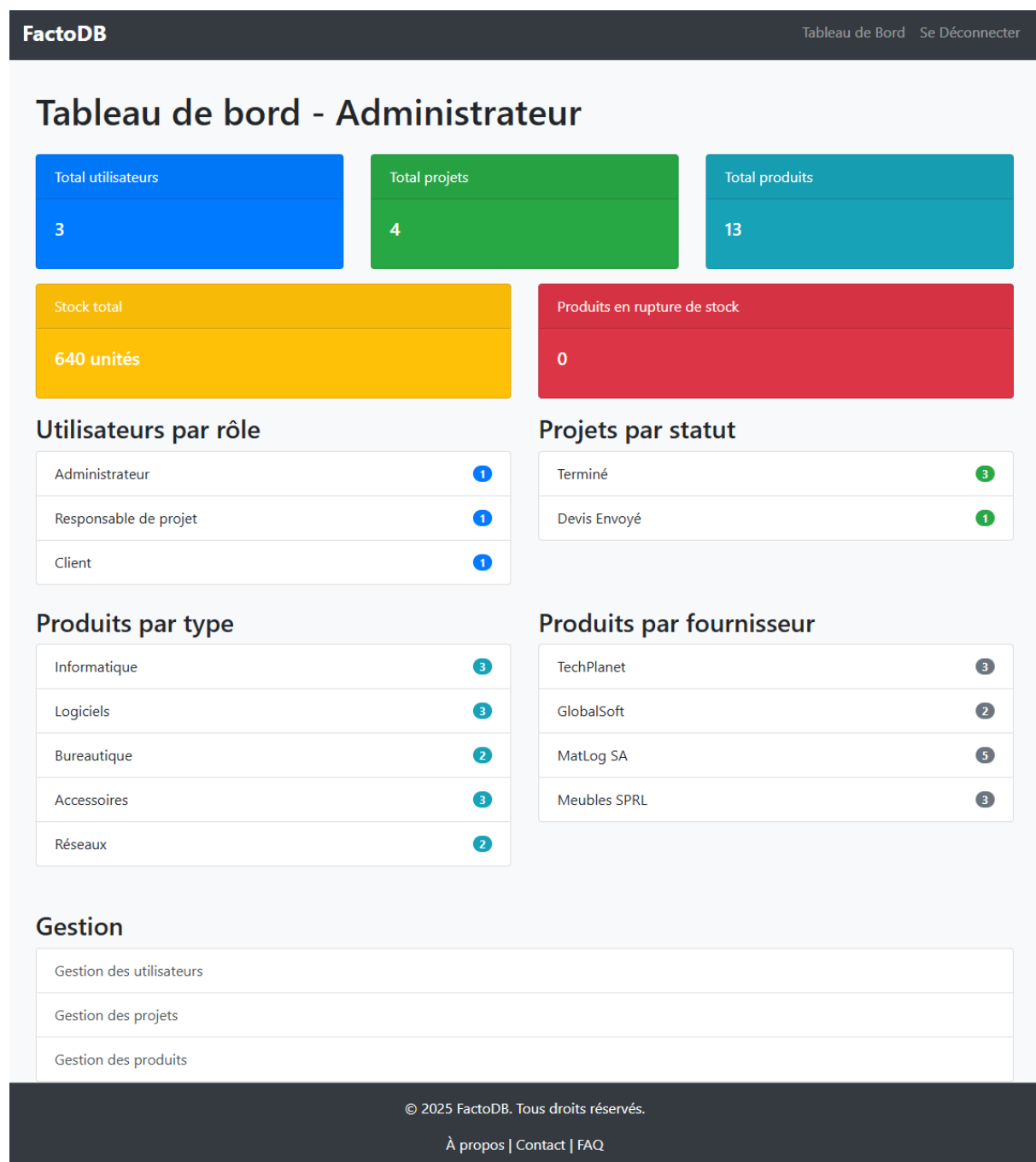
```
$stmt = $connection->prepare( query: "SELECT idUtilisateur FROM Utilisateur WHERE Email = ?");
```

- ❖ Requête **INSERT** pour insérer un nouvel utilisateur si l'email est libre :

```
$passwordHash = password_hash($password, algo: PASSWORD_BCRYPT);
$defaultRoleId = 3; // Client par défaut

$stmt = $connection->prepare( query: "
    INSERT INTO Utilisateur (Email, Mot_De_Passe_Hash, Nom, Prenom, Telephone, idRole)
    VALUES (?, ?, ?, ?, ?, ?)
");
$stmt->bind_param( types: "sssssi", &var1: $email, &var2: $passwordHash, $nom, $prenom, $telephone, $defaultRoleId);

if ($stmt->execute()) {
    $message = "Inscription réussie. Vous pouvez maintenant vous connecter.";
} else {
    $message = "Erreur lors de l'inscription : " . $stmt->error;
}
```



- ❖ Affiche des **statistiques** (totaux de projets, utilisateurs...), via des **SELECT** joins et agrégations (COUNT(*), GROUP BY...), par exemple :

```

22 // 2. Nombre total de projets
23 $stmt = $connection->prepare("SELECT COUNT(*) FROM Projet");
24 $stmt->execute();
25 $stmt->bind_result(&$var1, $total_projects);
26 $stmt->fetch();
27 $stmt->close();

```

Gestion des utilisateurs

Ajouter un nouvel utilisateur

Nom Prénom

Adresse email

Mot de passe

Rôle
Sélectionnez un rôle ▼

[Ajouter l'utilisateur](#)

Liste des utilisateurs

ID	Nom	Prénom	Email	Rôle	Actions
5	MANAGERTEST	Managertest	mng@mng.com	Responsable de projet	Éditer Supprimer
3	CLIENTTEST	Clienttest	client@client.com	Client	Éditer Supprimer
1	ADMIN	Admin	admin@admin.com	Administrateur	Éditer Supprimer

❖ Buts :

- Permettre à l'administrateur de **consulter** la liste de tous les utilisateurs.
- **Modifier** certains attributs de l'utilisateur (souvent le rôle).
- **Supprimer** un utilisateur, si nécessaire.

❖ Requête **SELECT** pour vérifier si l'utilisateur ajouté a déjà un compte avec cette adresse email :

```
// Vérifier si l'email existe déjà
$stmt = $connection->prepare( query: "SELECT idUtilisateur FROM Utilisateur WHERE Email = ?");
$stmt->bind_param( types: "s", &var1: $email);
$stmt->execute();
$stmt->store_result();
```

❖ Requête **INSERT INTO** pour ajouter un nouvel utilisateur :

```
// Insérer le nouvel utilisateur
$hashed_password = password_hash($password, algo: PASSWORD_BCRYPT);
$stmt_insert = $connection->prepare( query: "INSERT INTO Utilisateur (Nom, Prenom, Email, Password, idRole) VALUES (?, ?, ?, ?, ?)");
$stmt_insert->bind_param( types: "ssssi", &var1: $nom, &var2: $prenom, &var3: $email, &var4: $hashed_password, &var5: $role_id);
```

❖ Requête **UPDATE** pour modifier les informations de l'utilisateur :

```
// Mettre à jour les détails de l'utilisateur
if (!empty($new_password)) {
    // Si un nouveau mot de passe est fourni, le hacher
    $hashed_password = password_hash($new_password, algo: PASSWORD_BCRYPT);
    $stmt_update = $connection->prepare( query: "
        UPDATE Utilisateur
        SET Nom = ?, Prenom = ?, Email = ?, Password = ?, idRole = ?
        WHERE idUtilisateur = ?
    ");
```

Gestion des projets

Liste des Projets

ID	Nom du projet	Description	Statut	Date de début	Client	Responsables	Actions
4	Upgrade de l'IGLab 2	dzqjdohuqziphdqizd hiip	Devis Envoyé	08/01/2025	Clienttest CLIENTTEST	Managertest MANAGERTEST	Éditer Supprimer
3	Test projet	Test description	Terminé	13/12/2024	Clienttest CLIENTTEST	Managertest MANAGERTEST	Éditer Supprimer
2	Réchauffer	jj	Terminé	13/12/2024	Clienttest CLIENTTEST	Managertest MANAGERTEST	Éditer Supprimer
1	Upgrade de l'IGLab	Madame, Monsieur, Je me permets de vous contacter au nom de l'UMONS concernant notre salle inform...	Terminé	07/12/2024	Clienttest CLIENTTEST	Managertest MANAGERTEST	Éditer Supprimer

- ❖ Buts :
 - Récupération et affichage de la liste des projets et des informations complémentaires
 - Gestion de la modification et la suppression des projets
 - Présenter la liste de tous les projets en cours (ou terminés) avec leurs statuts.
 - Permettre l'édition ou la suppression d'un projet.
 - Gérer éventuellement l'affectation des responsables (jointure avec Projet_manager).
- ❖ Requête **DELETE** pour supprimer les relations du projet et le projet en lui-même :

```
// Supprimer les associations dans Projet_manager
$stmt = $connection->prepare("DELETE FROM Projet_manager WHERE idProjet = ?");
$stmt->bind_param("i", $delete_project_id);
if ($stmt->execute()) {
    $stmt->close();

    // Supprimer le projet
    $stmt = $connection->prepare("DELETE FROM Projet WHERE idProjet = ?");
    $stmt->bind_param("i", $delete_project_id);
    if ($stmt->execute()) {
        $delete_message = "Projet supprimé avec succès.";
        $delete_success = true;
    } else {
        $delete_message = "Erreur lors de la suppression du projet : " . htmlspecialchars($stmt->error);
    }
} else {
    $delete_message = "Erreur lors de la suppression des associations du projet : " . htmlspecialchars($stmt->error);
}
$stmt->close();
```

- ❖ Requête **SELECT** pour ressaisir la liste des projets avec toutes les informations (projets, responsables, clients...)

```

// Récupérer la liste des projets avec les informations du client et des responsables
$stmt = $connection->prepare("
    SELECT
        p.idProjet,
        p.Nom AS ProjetNom,
        p.Description,
        p.Statut,
        DATE_FORMAT(p.Date_Debut, '%d/%m/%Y') AS DateDebutFormat,
        u.Nom AS ClientNom,
        u.Prenom AS ClientPrenom
    FROM Projet p
    JOIN Utilisateur u ON p.idUtilisateur = u.idUtilisateur
    ORDER BY p.idProjet DESC
");
$stmt->execute();
$result = $stmt->get_result();
$projects = [];
while ($row = $result->fetch_assoc()) {
    // Récupérer les responsables pour chaque projet
    $project_id = $row['idProjet'];
    $stmt_managers = $connection->prepare("
        SELECT u.Nom, u.Prenom
        FROM Projet_manager pm
        JOIN Utilisateur u ON pm.idUtilisateur = u.idUtilisateur
        JOIN Role r ON u.idRole = r.idRole
        WHERE pm.idProjet = ? AND r.Description = 'Responsable de projet'
    ");
    $stmt_managers->bind_param("i", $project_id);
    $stmt_managers->execute();
    $result_managers = $stmt_managers->get_result();
    $managers = [];
    while ($manager = $result_managers->fetch_assoc()) {
        $managers[] = $manager['Prenom'] . ' ' . $manager['Nom'];
    }
    $row['Managers'] = $managers;
    $projects[] = $row;
    $stmt_managers->close();
}
$stmt->close();

```

```

// Récupérer la liste des responsables (managers) pour les assignations éventuelles
$stmt = $connection->prepare("
    SELECT u.idUtilisateur, u.Prenom, u.Nom
    FROM Utilisateur u
    JOIN Role r ON u.idRole = r.idRole
    WHERE r.Description = 'Responsable de projet'
    ORDER BY u.Prenom ASC, u.Nom ASC
");
$stmt->execute();
$result = $stmt->get_result();
$available_managers = [];
while ($row = $result->fetch_assoc()) {
    $available_managers[] = $row;
}
$stmt->close();

```

Gestion des produits

Ajouter un nouveau produit

Nom du produit

Description du produit

Prix (€)

Stock

Type de produit

Fournisseur

Ajouter le produit

Liste des Produits

ID	Nom	Description	Prix (€)	Stock	Type de Produit	Fournisseur	Actions
16	Chaise bureau	Chaise ergonomique pour bureau	300,00	50	Bureautique	Meubles SPRL	Éditer Supprimer
15	Bureau	Bureau en bois massif pour ordinateur fixe	700,00	20	Bureautique	Meubles SPRL	Éditer Supprimer
14	Routeur Wi-Fi Pro	Routeur Dual-Band haute performance	120,00	15	Réseaux	MatLog SA	Éditer Supprimer
13	Casque Audio	Casque pour visioconférence	25,00	80	Accessoires	TechPlanet	Éditer Supprimer
12	Logiciel CRM	CRM pour gestion de clients	300,00	25	Logiciels	GlobalSoft	Éditer Supprimer

❖ Buts :

- Afficher tous les produits (PC, licences, etc.), avec leurs stocks, leurs types et leurs fournisseurs.
- Permettre à l'administrateur de créer, modifier ou supprimer un produit.
- Gérer le stock et le prix.

- ❖ Requête **SELECT** pour afficher la liste des produits :

```
// Récupérer la liste des produits avec les informations du type de produit et du fournisseur
$stmt = $connection->prepare( query: "
SELECT
    p.idProduit,
    p.Nom AS ProduitNom,
    p.Description,
    p.Prix,
    p.Stock,
    tp.Nom AS TypeProduit,
    f.Nom AS Fournisseur
FROM Produit p
LEFT JOIN Type_produit tp ON p.idTypeProduit = tp.idTypeProduit
LEFT JOIN Fournisseur f ON p.idFournisseur = f.idFournisseur
ORDER BY p.idProduit DESC
");
```

- ❖ Requête **INSERT** pour ajouter un produit :

```
// Insérer le nouveau produit
$stmt = $connection->prepare("INSERT INTO Produit (Nom, Description, Prix, Stock, idTypeProduit, idFournisseur) VALUES (?, ?, ?, ?, ?, ?)");
$stmt->bind_param("ssdiili", $nom, $description, $prix, $stock, $type_produit_id, $fournisseur_id);
if ($stmt->execute()) {
    $add_message = "Produit ajouté avec succès.";
    $add_success = true;
} else {
    $add_message = "Erreur lors de l'ajout du produit : " . htmlspecialchars($stmt->error);
}
$stmt->close();
```

- ❖ Requête **UPDATE** (dans admin/edit_product.php) pour modifier un produit :

```
// Mettre à jour le produit
$stmt = $connection->prepare("
UPDATE Produit
SET Nom = ?, Description = ?, Prix = ?, Stock = ?, idTypeProduit = ?, idFournisseur = ?
WHERE idProduit = ?
");
$stmt->bind_param("ssdiili", $new_nom, $new_description, $new_prix, $new_stock, $new_type_produit_id, $new_fournisseur_id, $product_id);
if ($stmt->execute()) {
    $edit_message = "Produit mis à jour avec succès.";
    $edit_success = true;

    // Mettre à jour les variables avec les nouvelles valeurs pour l'affichage
    $nom = $new_nom;
    $description = $new_description;
    $prix = $new_prix;
    $stock = $new_stock;
    $type_produit_id = $new_type_produit_id;
    $fournisseur_id = $new_fournisseur_id;
} else {
    $edit_message = "Erreur lors de la mise à jour du produit : " . htmlspecialchars($stmt->error);
}
$stmt->close();
```

- ❖ Requête **DELETE** pour supprimer un produit :

```
// Traitement de la suppression d'un produit
$delete_message = '';
$delete_success = false;
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action']) && $_POST['action'] === 'delete_product') {
    $delete_product_id = intval($_POST['product_id']);

    // Supprimer le produit
    $stmt = $connection->prepare("DELETE FROM Produit WHERE idProduit = ?");
    $stmt->bind_param("i", $delete_product_id);
    if ($stmt->execute()) {
        $delete_message = "Produit supprimé avec succès.";
        $delete_success = true;
    } else {
        $delete_message = "Erreur lors de la suppression du produit : " . htmlspecialchars($stmt->error);
    }
    $stmt->close();
}
```


Tableau de bord - Manager

Mes Projets

ID	Nom du projet	Description	Statut	Date de début	Actions
4	Upgrade de l'IGLab 2	dzqjdohuqzipdpqzdzd hiip	Devis Envoyé	08/01/2025	Accéder au Projet
3	Test projet	Test description	Terminé	13/12/2024	Accéder au Projet
2	Réchauffer	jj	Terminé	13/12/2024	Accéder au Projet
1	Upgrade de l'IGLab	Madame, Monsieur, Je me permets de vous contacter au nom de l'UMONS concernant notre salle inform...	Terminé	07/12/2024	Accéder au Projet

❖ Buts :

- Présenter la liste des projets assignés à cet utilisateur avec le rôle « Responsable de projet ».
- Offrir un accès rapide à chaque projet pour le consulter ou le modifier.

❖ Requête **SELECT** pour récupérer les projets :

```
// Récupérer la liste des projets assignés au manager
$stmt = $connection->prepare( query: "
    SELECT
        p.idProjet,
        p.Nom AS ProjetNom,
        p.Description,
        p.Statut,
        DATE_FORMAT(p.Date_Debut, '%d/%m/%Y') AS DateDebutFormat
    FROM Projet p
    JOIN Projet_manager pm ON p.idProjet = pm.idProjet
    WHERE pm.idUtilisateur = ?
    ORDER BY p.idProjet DESC
");
```

Gestion du projet

Modifier les détails du projet

Nom du projet

Upgrade de l'IGLab

Statut du projet

Terminé

Date de début

07/12/2024

Description du projet

Madame, Monsieur,

Je me permets de vous contacter au nom de l'UMONS concernant notre salle informatique, le "IGLab". Nous souhaitons améliorer cette salle afin d'optimiser son fonctionnement et de proposer un environnement plus moderne et performant à nos utilisateurs.

Mettre à jour le projet

Panier

Nom du produit	Prix (€)	Quantité	Total (€)	Enlever du panier	
PC Portable	700,00	6	4 200,00	1	Enlever
Casque Audio	25,00	5	125,00	1	Enlever
Montant Total :			4 325,00 €		

Valider et envoyer le devis au client

Ajouter des produits au panier

Nom	Description	Prix (€)	Stock	Ajouter au panier	
Bureau	Bureau en bois massif pour ordinateur fixe	700,00	20	1	Ajouter
Casque Audio	Casque pour visioconférence	25,00	75	1	Ajouter
Chaise bureau	Chaise ergonomique pour bureau	300,00	50	1	Ajouter
Clavier Mécanique	Clavier mécanique haute durabilité	70,00	45	1	Ajouter
Logiciel CRM	CRM pour gestion de clients	300,00	25	1	Ajouter

❖ Buts :

- Afficher tous les détails du projet sélectionné, y compris :
- Nom, description, date de début, statut...
 - Possibilité de modifier le projet (nom, description, dates...).
 - Présenter un Panier associé au projet, permettant d'ajouter des produits/services et de générer un devis.
- Gérer des commentaires du client ou des collaborateurs.

❖ Requête **SELECT** pour vérifier que le projet appartient au manager connecté :

```
// Vérifier que le manager est assigné à ce projet
$stmt = $connection->prepare( query: "
SELECT p.idProjet, p.Nom, p.Description, p.Statut, p.Date_Debut, p.idPanier
FROM Projet p
JOIN Projet_manager pm ON p.idProjet = pm.idProjet
WHERE p.idProjet = ? AND pm.idUtilisateur = ?
");
```

❖ Requête **UPDATE** pour modifier les détails d'un projet :

```
// Mettre à jour le projet
$stmt = $connection->prepare( query: "
UPDATE Projet
SET Nom = ?, Description = ?, Statut = ?, Date_Debut = ?
WHERE idProjet = ?
");
$stmt->bind_param( types: "ssssi", &var1: $new_nom, &var2: $new_description, &var3: $new_statut, &var4: $new_date_debut, &var5: $project_id);
```

❖ Requête **SELECT** pour afficher les commentaires :

```
// Récupérer les commentaires du client pour ce projet
$stmt = $connection->prepare( query: "
SELECT c.Commentaire, c.Date_Commentaire, u.Nom, u.Prenom
FROM Commentaires c
JOIN Utilisateur u ON c.idUtilisateur = u.idUtilisateur
WHERE c.idProjet = ?
ORDER BY c.Date_Commentaire ASC
");
```

❖ Requête **INSERT INTO** et **UPDATE** pour ajouter un produit dans le panier :

```
if (is_null($panier_id)) {
    // Créer un nouveau panier
    $stmt = $connection->prepare( query: "INSERT INTO Panier (Date_Panier, Montant, Statut) VALUES (CURDATE(), 0, 'En cours')");
    if ($stmt->execute()) {
        $panier_id = $stmt->insert_id;
        // Mettre à jour le projet avec le nouveau panier
        $stmt_update = $connection->prepare( query: "UPDATE Projet SET idPanier = ? WHERE idProjet = ?");
        $stmt_update->bind_param( types: "ii", &var1: $panier_id, &var2: $project_id);
        $stmt_update->execute();
        $stmt_update->close();
    }
    $stmt->close();
}
```

- ❖ Requête **UPDATE** puis **DELETE** pour retirer un produit du panier et le supprimer lorsque son stock devient nul :

```
if ($nouvelle_quantite > 0) {
    // Mettre à jour la quantité dans le panier si elle reste positive
    $stmt = $connection->prepare( query: "UPDATE Panier_produit SET Quantite = ? WHERE idPanier = ? AND idProduit = ?");
    $stmt->bind_param( types: "iii", &var1: $nouvelle_quantite, &var2: $panier_id, $product_id);
    if ($stmt->execute()) {
        // $edit_message = "Quantité mise à jour dans le panier.";
    } else {
        $edit_message = "Erreur lors de la mise à jour du panier : " . htmlspecialchars($stmt->error);
    }
    $stmt->close();
} else {
    // Supprimer l'entrée du panier si la quantité atteint 0
    $stmt = $connection->prepare( query: "DELETE FROM Panier_produit WHERE idPanier = ? AND idProduit = ?");
    $stmt->bind_param( types: "ii", &var1: $panier_id, &var2: $product_id);
    if ($stmt->execute()) {
        // $edit_message = "Produit retiré du panier.";
    } else {
        $edit_message = "Erreur lors de la suppression du produit du panier : " . htmlspecialchars($stmt->error);
    }
    $stmt->close();
}
```

CLIENT : PAGE D'ACCUEIL (CLIENT/DASHBOARD_CLIENT.PHP)

FactoDB
Tableau de Bord Se Déconnecter

Bienvenue, Clienttest CLIENTTEST!

Accédez à vos services et fonctionnalités personnalisées ci-dessous.

Vous pouvez consulter vos projets ou en créer un nouveau.

Consulter mes projets
Créer un projet

- ❖ Buts :
 - Accueillir l'utilisateur identifié comme Client.
 - Proposer des actions : créer un nouveau projet, consulter ses projets existants.
 - Afficher un bref résumé (nombre de projets, leur statut...).
- ❖ Requête **SELECT** pour saisir les informations de l'utilisateur à l'écran

```
// Récupérer les informations de l'utilisateur pour affichage
$stmt = $connection->prepare("SELECT Nom, Prenom FROM Utilisateur WHERE idutilisateur = ?");
$stmt->bind_param("i", $user_id);
$stmt->execute();
$stmt->bind_result($nom, $prenom);
$stmt->fetch();
$stmt->close();
$connection->close();
```

FactoDB
Tableau de Bord Se Déconnecter

Créer un nouveau projet

Nom du projet

Description du projet

Créer le projet

- ❖ Buts :
 - Permettre au client de définir un nom et une description pour son nouveau projet.
 - Enregistrer ce projet dans la base de données, éventuellement avec un statut « En cours de validation ».
- ❖ Requête **INSERT** pour insérer les informations sur le projet créé dans la BDD

```
// Insertion du projet dans la base
// Statut = "En cours de validation"
// Date_Debut = CURDATE()
$stmt = $connection->prepare("
    INSERT INTO Projet (Nom, Description, Date_Debut, Statut, idUtilisateur)
    VALUES (?, ?, CURDATE(), 'En cours de validation', ?)
");
if ($stmt) {
    $stmt->bind_param("ssi", $nom_projet, $description, $user_id);
    if ($stmt->execute()) {
        $message = "Votre projet a été créé avec succès et est en cours de validation par un administrateur.";
        $success = true;
    } else {
        $message = "Erreur lors de la création du projet : " . htmlspecialchars($stmt->error);
    }
    $stmt->close();
} else {
    $message = "Erreur lors de la préparation de la requête : " . htmlspecialchars($connection->error);
}
```

FactoDB
Tableau de Bord Se Déconnecter

Mes Projets

Retrouvez ici la liste de tous vos projets, avec leur statut actuel.

Nom	Description	Date de début	Statut	Action
Upgrade de l'IGLab 2	dzqjdohuqzphdpqizd hiip...	08/01/2025	Devis Envoyé	Accéder au projet
Test projet	Test description...	13/12/2024	Terminé	Accéder au projet
Réchauffer	jj...	13/12/2024	Terminé	Accéder au projet
Upgrade de l'IGLab	Madame, Monsieur, Je me permets de vous contacter au nom de l'UMONS concernant notre salle inform...	07/12/2024	Terminé	Accéder au projet

[Créer un nouveau projet](#)
[Retour au tableau de bord](#)

❖ Buts :

- Afficher tous les projets créés par ou affectés à ce client.
- Permettre de cliquer sur un projet pour accéder à ses détails (statut, description, etc.).

❖ Requête **SELECT** récupérer la liste des projets du client

```
// Récupérer la liste des projets de l'utilisateur
$stmt = $connection->prepare("
    SELECT idProjet, Nom, Description, Statut, DATE_FORMAT(Date_Debut, '%d/%m/%Y') as DateDebutFormat
    FROM Projet
    WHERE idUtilisateur = ?
    ORDER BY idProjet DESC
");
$stmt->bind_param("i", $user_id);
$stmt->execute();
$result = $stmt->get_result();

$projects = [];
while ($row = $result->fetch_assoc()) {
    $projects[] = $row;
}
$stmt->close();
$connection->close();
```

Upgrade de l'IGLab

Description : Madame, Monsieur,

Je me permets de vous contacter au nom de l'UMONS concernant notre salle informatique, le "IGLab". Nous souhaitons améliorer cette salle afin d'optimiser son fonctionnement et de proposer un environnement plus moderne et performant à nos utilisateurs.

Nous serions intéressés par une offre complète incluant, mais sans s'y limiter :

La mise à jour du matériel informatique (ordinateurs, serveurs, périphériques).

L'amélioration des infrastructures réseau (switchs, câblage, sécurité).

L'intégration de solutions logicielles adaptées à nos besoins.

Un éventuel accompagnement pour la formation des utilisateurs et la gestion technique de l'espace.

Afin de nous aider à définir précisément notre projet, nous serions ravis de recevoir une proposition détaillée, incluant vos conseils techniques, les équipements que vous recommandez, ainsi qu'une estimation des coûts associés.

Restant à votre disposition pour toute information complémentaire ou pour organiser une visite de la salle, je vous remercie par avance de votre attention et de votre réactivité.

Dans l'attente de votre retour, je vous prie d'agréer, Madame, Monsieur, l'expression de mes salutations distinguées.

Date de début : 07/12/2024

Statut : Terminé

Responsable(s) : Managertest MANAGERTEST

Commentaires

Ajouter un commentaire

Ajouter

Clienttest CLIENTTEST

Test

2024-12-07 14:33:37

Devis

Nom du produit	Prix (€)	Quantité	Total (€)
PC Portable	700,00	6	4 200,00
Casque Audio	25,00	5	125,00
Montant total :			4 325,00 €

Vous avez accepté le devis. Le projet est maintenant terminé.

Voici votre facture (PDF) :

[Télécharger la Facture \(PDF\)](#)

Veuillez effectuer le paiement par virement bancaire sur le compte suivant :

- **IBAN** : BE76 1234 5678 9012
- **BIC** : ABCD1234XXX
- **Commentaire** : 3-1

Merci de votre confiance.

❖ Buts :

- Permettre au client de consulter les informations du projet, son statut, ses dates, etc.
- Afficher les commentaires, ou laisser le client en ajouter.
- Afficher le devis (Panier) si un manager l'a envoyé.
- Permettre au client d'accepter ou de refuser le devis (ce qui met à jour le statut du projet).

❖ Requête SELECT pour afficher les commentaires :

```
// Récupérer les commentaires du projet
$comments = [];
$stmt = $connection->prepare("
    SELECT c.Commentaire, c.Date_Commentaire, u.Nom, u.Prenom
    FROM Commentaires c
    JOIN Utilisateur u ON c.idUtilisateur = u.idUtilisateur
    WHERE c.idProjet = ?
    ORDER BY c.Date_Commentaire DESC
");
$stmt->bind_param("i", $idProjet);
$stmt->execute();
$result = $stmt->get_result();
while ($row = $result->fetch_assoc()) {
    $comments[] = $row;
}
$stmt->close();
```


- ❖ Requête INSERT INTO pour ajouter un commentaire :

```
// Ajout de commentaire
$comment = trim($_POST['comment']);
if (!empty($comment)) {
    $stmt = $connection->prepare( query: "
        INSERT INTO Commentaires (idProjet, idUtilisateur, Commentaire, Date_Commentaire)
        VALUES (?, ?, ?, NOW())
    ");
    if ($stmt) {
        $stmt->bind_param( types: "iis", &var1: $idProjet, &var2: $user_id, $comment);
        if ($stmt->execute()) {
            $message = "Votre commentaire a été ajouté avec succès.";
            $success = true;
        }
    }
}
```

- ❖ Requête UPDATE pour modifier le statut du projet :

```
// Accepter ou refuser le devis
if ($project['Statut'] === 'Devis Envoyé') {
    $nouveau_statut = ($action === 'accept_devis') ? 'Terminé' : 'Refusé';

    $stmt = $connection->prepare( query: "UPDATE Projet SET Statut = ? WHERE idProjet = ?");
    $stmt->bind_param( types: "si", &var1: $nouveau_statut, &var2: $idProjet);
    if ($stmt->execute()) {
        $message = $action === 'accept_devis'
            ? "Vous avez accepté le devis. Le projet est désormais " . $nouveau_statut . "."
            : "Vous avez refusé le devis. Le projet est désormais " . $nouveau_statut . ".";
        $success = true;
        $project['Statut'] = $nouveau_statut;
    } else {
        $message = "Erreur lors de la mise à jour du statut du projet : " . htmlspecialchars($stmt->error);
    }
    $stmt->close();
}
```

Facture

Projet : Upgrade de l'IGLab

Description : Madame, Monsieur,

Je me permets de vous contacter au nom de l'UMONS concernant notre salle informatique, le "IGLab". Nous souhaitons améliorer cette salle afin d'optimiser son fonctionnement et de proposer un environnement plus moderne et performant à nos utilisateurs.

Nous serions intéressés par une offre complète incluant, mais sans s'y limiter :

La mise à jour du matériel informatique (ordinateurs, serveurs, périphériques).
L'amélioration des infrastructures réseau (switchs, câblage, sécurité).
L'intégration de solutions logicielles adaptées à nos besoins.
Un éventuel accompagnement pour la formation des utilisateurs et la gestion technique de l'espace.
Afin de nous aider à définir précisément notre projet, nous serions ravis de recevoir une proposition détaillée, incluant vos conseils techniques, les équipements que vous recommandez, ainsi qu'une estimation des coûts associés.

Restant à votre disposition pour toute information complémentaire ou pour organiser une visite de la salle, je vous remercie par avance de votre attention et de votre réactivité.

Dans l'attente de votre retour, je vous prie d'agréer, Madame, Monsieur, l'expression de mes salutations distinguées.

Date de Début : 07/12/2024

Nom du Produit	Prix Unitaire (€)	Quantité	Total (€)
PC Portable	700,00	6	4 200,00
Casque Audio	25,00	5	125,00

Total : 4 325,00 €

Informations de Paiement

Veuillez effectuer le paiement par virement bancaire sur le compte suivant :

- **IBAN :** BE76 1234 5678 9012
- **BIC :** ABCD1234XXX

Merci de votre confiance.

❖ Requête **SELECT** pour afficher les détails du projet, ainsi que le panier :

```
// Récupérer les détails du projet
$stmt = $connection->prepare( query: "
    SELECT p.Nom, p.Description, p.Statut, DATE_FORMAT(p.Date_Debut, '%d/%m/%Y') AS DateDebutFormat, p.idPanier
    FROM Projet p
    WHERE p.idProjet = ? AND p.idUtilisateur = ?
");

// Récupérer les produits du panier
$stmt = $connection->prepare( query: "
    SELECT p.Nom, p.Prix, pp.Quantite
    FROM Panier_produit pp
    JOIN Produit p ON pp.idProduit = p.idProduit
    WHERE pp.idPanier = ?
");
```

CONCLUSION

Au terme de ce projet, nous avons pu mettre en pratique l'ensemble des notions abordées dans le cadre du cours, tout en répondant à des besoins concrets.

L'établissement d'un cahier de charges détaillé a permis de clarifier les attentes du « client » et de fixer le périmètre du projet. Cette étape initiale a été cruciale : en définissant précisément les objectifs et contraintes, nous avons réduit les risques de malentendus et pu structurer efficacement notre travail.

La conception du MCD nous a amenés à réfléchir aux entités, attributs et relations clés de l'application (utilisateurs, projets, produits, etc.). La normalisation (jusqu'à la troisième forme normale) a consolidé ce travail, nous assurant une base solide où les redondances sont limitées et l'intégrité des données préservée.

Sur base du MCD normalisé, la traduction en MLD nous a fait entrer dans une vision plus technique des relations et clés étrangères. Cela a consolidé notre compréhension sur la structure relationnelle et la manière dont chaque entité s'articule avec les autres (par exemple, la table Panier_produit pour lier la quantité aux produits et la normaliser, ou encore la gestion de rôles via Role).

La réalisation du MPD (création des tables, types de champs, contraintes) nous a permis de finaliser la base de données dans un système concret (MySQL), assurant que la structure respecte la logique définie et qu'elle peut être réellement implémentée.

L'application finale combine un socle PHP/MySQL, enrichi par HTML, CSS, JavaScript et Bootstrap pour l'interface graphique, ainsi que des bibliothèques comme DataTables et Dompdf. Les requêtes SQL (préparées au préalable) gèrent chaque interaction critique (authentification, création de projets, gestion du panier, validation de devis, etc.). La séparation claire des rôles (administrateur, manager, client) et la mise en place de formulaires sécurisés en PHP nous ont conduits à une application complète et robuste.

En conclusion, ce travail nous a offert une expérience pratique et transversale : l'utilisation d'une base de données sous SQL, à la normalisation d'un modèle conceptuel à la prise en main de technologies et à la sécurisation via des requêtes préparées. Nous avons ainsi concrétisé la théorie du cours en un projet complet, répondant aux besoins identifiés, et offrant à la fois une structure de base de données solide et une interface adaptée aux différents rôles.